Architecture and Implementation of an FPGA-Based CNN Accelerator Using Hardware–Software Co-Design

A report submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

by

Tejo Priya Boddu
ID: N200429

Under the supervision of

MR.SIVALAL KETHAVATH

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES

NUZVID, INDIA

DECEMBER 10,2025

DECLARATION

I, **Tejo Priya Boddu**, hereby declare that the project report is submitted for partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Electronics and Communication Engineering of the **Rajiv Gandhi University of Knowledge Technologies, Nuzvid,** is a bonafide work done by me under (supervision under) the guidance of **MR.SIVALAL KETHAVATH**. This submission represents our ideas in our own words, and where ideas or words of others have been included, We have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact or source in my submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the university and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously been formed as the basis for the award of any degree, diploma, or similar title from any other university.

Tejo Priya Boddu(N200429)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES, NUZVID, INDIA

CERTIFICATE

This is to certify that the report entitled **"Architecture and Implementation of an FPGA-Based CNN Accelerator Using Hardware–Software Co-Design"** submitted by **Tejo Priya Boddu (N200429)** to the Rajiv Gandhi University of Knowledge Technologies, Nuzvid in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Electronics and Communication Engineering is a bonafide record of the project work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Guide
MR.Sivalal K Sir
Assistant Professor
Dept. of Electronics and Communication Engineering
RGUKT, NUZVID CAMPUS
Nuzvid-521202

Head of the Department
Mr. Vinod Babu Sir
Assistant Professor
Dept. of Electronics and Communication Engineering
RGUKT NUZVID CAMPUS
Nuzvid-521202

ACKNOWLEDGMENT

I express my sincere gratitude to our guide, Sivalal Kethavath , Electronics and Communication Engineering, Rajiv Gandhi University of Knowledge Technologies, Nuzvid, for his valuable guidance, suggestions, and supervision throughout the work. Without his patronage and guidance, the project would not have taken shape. We also express our appreciation for your kind approval of the project, time-to-time counseling, and advice. I owe my sincere thanks to the department of Electronics and Communication Engineering, Rajiv Gandhi University of Knowledge Technologies, Nuzvid.

ABSTRACT

This project presents the design, implementation, and rigorous validation of a high-performance Deep Neural Network (DNN) Inference Engine that targets embedded applications on a Xilinx Artix-7 Field-Programmable Gate Array (FPGA). The primary objective was to demonstrate a highly efficient alternative to traditional processors for computationally intensive convolutional and activation operations.

The system utilizes a Heterogeneous System-on-Chip (SoC) Architecture centered around a MicroBlaze Soft Processor for system control and communication, tightly coupled with a specialized, custom Convolutional Neural Network (CNN) accelerator IP. This accelerator hardware was automatically generated from an optimized C/C++ model using High-Level Synthesis (HLS), allowing the design to exploit massive hardware parallelism.

The integrated design was fully implemented with the Vivado Implementation flow (Synthesis, Placement, and Routing). The project ends after generating the post-route reports, which are used to validate the technical success of the design.

Critical post-implementation analysis confirmed:

- Guaranteed Performance: The design successfully met all internal timing requirements, confirming reliable operation at the target clock frequency.
- Resource Efficiency: The entire system, including the MicroBlaze core and accelerator, was efficiently mapped and routed, demonstrating optimized utilization of the Artix-7's onboard resources (Block RAM, LUTs, and DSPs).
- Low Power Feasibility: The architecture exhibited power characteristics suitable for deployment in edge and power-constrained embedded systems.

This work establishes a verified hardware foundation, ready for the final step of generating the bitstream and moving to the Vitis platform for high-level application development and end-to-end functional testing.

Contents

1 INTRODUCTION **8**

1.1 Background 8

1.2 Objective of the project 8

1.3 Scope of the project 9

2 LITERATURE REVIEW **10**

2.1 Evolution of Deep Learning Acceleration 10

2.2 The High-Level Synthesis (HLS) Paradigm Shift 11

2.3 Limitations of Prior Work and Research Gaps 11

2.4 Project Contribution 12

3 METHODOLOGY **13**

3.1 System Design and Workflow 13

3.2 System Architecture 14

3.2.1 The Processing System 14

3.2.2 The Custom Accelerator System 14

3.3 Working Principle: 15

3.3.1 Phase 1: Software Configuration and Data Preparation 15

3.3.2 Phase 2: Hardware Computation and Data Movement 15

3.3.3 Phase 3: Hardware/Software Synchronization 15

3.3.4 Phase 4: Software Post-Processing and Output 16

3.4 System Requirements 16

3.4.1 Hardware System Requirements 16

3.4.2 Software System Requirements 31

4 Implementation Process **32**

4.1 C-Level Model Creation 32

4.2 C/C++ Simulation (CSIM) 32

4.3 HLS Synthesis and Optimization 33

4.4 Export RTL IP Core 34

4.5 Create Block Diagram in Vivado 35

4.6 Peripheral Configuration 36

4.7 Generate HDL Wrapper 37
4.8 Implementation (Synthesis → Placement → Routing) 38
4.9 Post-Route Analysis 38
4.10 Packaging 39

5 Results **40**
5.1 Floorplanning Results 40
5.2 Resource Utilization Results 42
5.3 Routing Results 43
5.4 Timing Analysis Results 45
5.5 Power Analysis Results 46
5.6 RTL Schematic Results 47
5.7 IP Packaging Results 48
5.8 Overall Implementation Summary 49
5.9 Final Result Statement 49

6 DISCUSSIONS **50**

7 CONCLUSION **52**

8 REFERENCES **53**

List of Figures

3.1 MicroBlaze IP 17
3.2 Custom CNN Accelerator 20
3.3 AXI Interconnect Diagram 24
3.4 AXI BRAM Controller and Block RAM 26
3.5 AXI UART Lite IP 29

4.1 Initial Synsthesis 33
4.2 Optimization - 1 33
4.3 Final Optimization 34
4.4 Block Diagram 35
4.5 Address Editor 36
4.6 Implementation 38

5.1 Floor Planning a 41
5.2 Floor Planning b 41
5.3 Resource Utilization Report 42
5.4 Routing Console Output 43
5.5 Routing a 44
5.6 Routing b 44
5.7 Timing Reports 45
5.8 Power Report 46
5.9 Power Utilization 47
5.10 RTL Schematic 48
5.11 IP Package 49

Chapter 1

INTRODUCTION

1.1 Background

The exponential growth of Artificial Intelligence (AI) and Deep Learning (DL) has driven a critical demand for high-performance computing solutions capable of executing complex neural network models efficiently. Specifically, Convolutional Neural Networks (CNNs)—the foundation for tasks such as image recognition and computer vision—require massive parallel arithmetic computation. Although Graphics Processing Units (GPUs) offer high throughput, they often consume significant power and are not ideal for edge computing or embedded systems where power and latency are constrained.

Field-Programmable Gate Arrays (FPGAs) provide a superior alternative in this domain. Their reconfigurable nature allows for the creation of custom, parallel hardware pipelines directly optimized for CNN layer operations (convolution, pooling, and activation). This capability offers a powerful combination of high performance, low power consumption, and deterministic low latency execution, making them ideal for accelerating DNN inference at the edge.

1.2 Objective of the project

The primary objective of this project is to successfully design, implement and validate a hybrid System-on-Chip (SoC) architecture for Deep Neural Network (DNN) acceleration following a rigorous, systematic hardware development flow.
The specific objectives are :

- C-Level Functional Verification: Perform C Simulation and C Synthesis using the High-Level Synthesis tool (HLS) to verify the functional correctness and establish the performance baseline of the conv2d algorithm.

- IP Core Generation: Generate and export the validated conv2d algorithm as a custom AXI4-Lite compliant Intellectual Property (IP) core, making it ready for integration into a processor-based system.
- MicroBlaze Integration: Create the complete Heterogeneous SoC Architecture in Vivado IP Integrator, connecting the MicroBlaze Soft Processor, necessary AXI peripherals (UART, Interrupt Controller), and the custom CNN Accelerator IP via the AXI Interconnect.
- Hardware Synthesis and Optimization: Successfully run the design through the Synthesis and Logic Optimization phases to translate the register-transfer-level (RTL) netlist into an optimized hardware description ready for physical placement.
- Physical Implementation and Timing Closure: Complete the Placement and Routing phases on the target Artix-7 XC7A100T device, ensuring that all critical data paths are physically routed and successfully meet the specified timing constraints.
- Performance and Utilization Reporting: Generate and analyze the final post-route reports, including the Timing Summary Report and Resource Utilization Report, to formally document and validate the design's guaranteed performance and resource efficiency.

1.3 Scope of the project

The scope of this project is explicitly defined as the completion and technical validation of the entire hardware design flow in the Xilinx Vivado environment. The project encompasses the following stages:

- Design Creation: Implement the entire SoC architecture, including the MicroBlaze processor, on-chip memory, AXI Interconnect, and the custom conv2d_accelerator IP.
- Implementation: Run the full FPGA compilation flow (Synthesis, Optimization, Placement, and Routing) targeting the Artix-7 XC7A100T device.
- Documentation: Generating and analyzing critical post-implementation reports, specifically the Timing Summary and Resource Utilization reports, to formally document the technical success and performance metrics of the hardware.

The project consciously concludes at the point where these reports fully validate the fully routed design checkpoint validated by these reports. The subsequent stages—writing the C-level driver software in Vitis and testing the end-to-end inference on the physical hardware—are defined as Future Work based on the success of this implemented hardware foundation.

Chapter 2

LITERATURE REVIEW

2.1 Evolution of Deep Learning Acceleration

The intense computational demands of Deep Neural Networks (DNNs), particularly Convolutional Neural Networks (CNNs), have historically driven the need for dedicated hardware acceleration.
This led to a paradigm shift toward specialized hardware:

- GPUs Dominance: Initially, Graphics Processing Units (GPUs) became the dominant platform. Architectures like NVIDIA's CUDA provided high throughput due to their massive floating-point parallelism, making them ideal for training large models in data centers. However, this approach is characterized by high power consumption and limited control over data movement, which severely restricts deployment in power-constrained edge and embedded systems.

- ASICs Specialization: At the other extreme, Application-Specific Integrated Circuits (ASICs), such as Google's Tensor Processing Unit (TPU), offer the highest performance and energy efficiency by hard-coding the network architecture. This comes at the cost of zero flexibility and high Non-Recurring Engineering (NRE) costs, making them impractical for research and evolving algorithm deployment.

- FPGAs as a Compromise: Field-Programmable Gate Arrays (FPGAs) emerged as anoptimal compromise. They offer a reconfigurable fabric that can create highly parallel, low-latency, and power-efficient custom data paths, placing them perfectly in the edge computing niche. Early FPGA designs relied on manual Hardware Description Languages (HDL) such as VHDL or Verilog, which required extensive expertise and slowed the development cycle significantly.

2.2 The High-Level Synthesis (HLS) Paradigm Shift

The complexity of manual HDL design was addressed by High-Level Synthesis (HLS) tools (e.g., Xilinx Vitis HLS). HLS allows for the automatic translation of verified C/C++ code into optimized Register-Transfer Level (RTL) hardware. This innovation accelerated the development time and enabled the adoption of the Hardware/Software Co-Design paradigm:

- Heterogeneous SoC Architectures: Current research favors building System-on-Chip (SoC) platforms, where a soft processor (like MicroBlaze) handles general-purpose tasks (boot-up, configuration, I/O), and the computationally intensive kernels (conv2d activation) are offloaded to custom HLS-generated accelerators connected via the AXI bus (Advanced eXtensible Interface).

2.3 Limitations of Prior Work and Research Gaps

Despite the significant advances offered by HLS and SoC integration, several key limitations remain in the published literature, which this project addresses:

Table 2.1: Limitations in Prior Work and Their Impact

Category	Limitation in Prior Work	Impact
System Validation	Many published works prioritize the C-level algorithmic performance (latency/throughput) of the accelerator IP in isolation.	This approach often fails to account for the actual overhead and complexity introduced when integrating the IP core into a full processor-based system (MicroBlaze and AXI Interconnect).
Physical Implementation	Focus is frequently on pre-synthesis or high-level resource estimates, with incomplete documentation of the physical implementation results.	Timing closure (guaranteeing performance after physical routing) is often assumed, but rarely proven with concrete Worst Negative Slack (WNS) reports, leaving the design's reliability uncertain.
Resource Context	Resource utilization (BRAM, DSP) is often reported for the accelerator IP only, divorced from the full SoC context.	This obscures the total resource budget required for the entire embedded system (including MicroBlaze, memory, and AXI fabric), which is critical for resource-constrained FPGAs like the Artix-7.

2.4 Project Contribution

This project is specifically designed to address these gaps by providing a fully implemented and technically validated solution on the Artix-7 platform. Our work constitutes the following distinct contributions

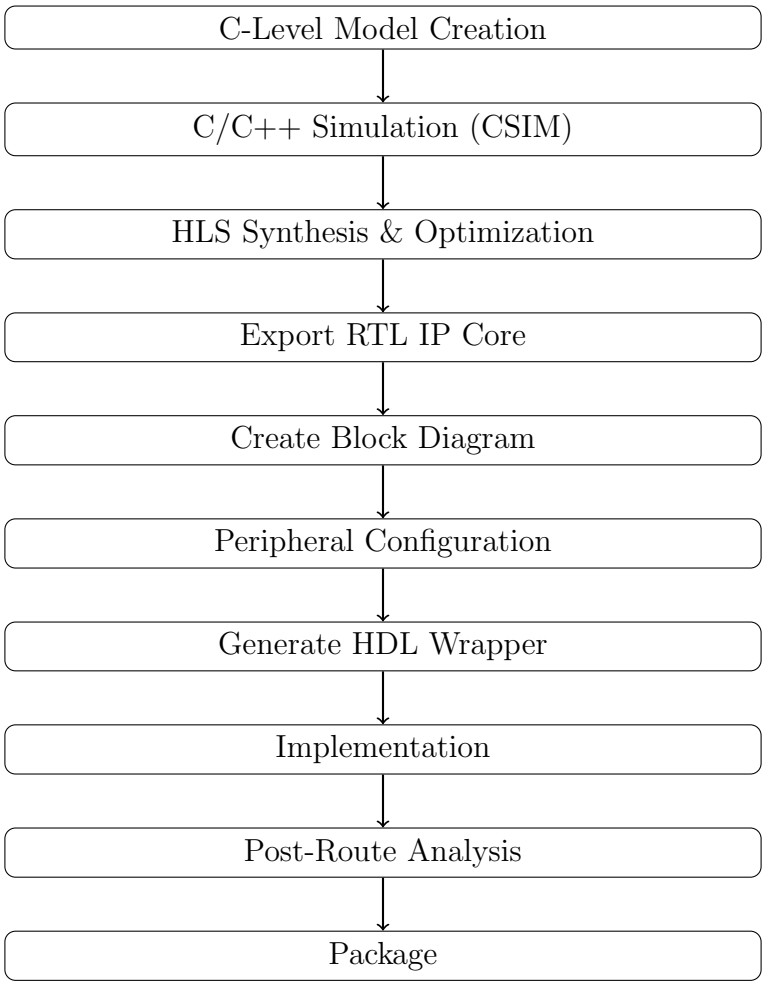
Full System Design Closure: We move beyond the HLS phase to deliver and document the successful integration and compilation of the complete MicroBlaze-AXI Accelerator SoC architecture.
Guaranteed Hardware Performance: We explicitly provide the post-route Timing Summary Report, serving as definitive proof that the design has achieved timing closure and will operate reliably at the target frequency on the specified hardware.
Comprehensive Resource Benchmarking: We document the final utilization and power consumption of the entire routed SoC, establishing a clear and reliable benchmark for the deployment of complex DNN inference engines on medium-scale, power-efficient Artix-7 devices.

Chapter 3

METHODOLOGY

3.1 System Design and Workflow

This is the two-stage technical workflow used to design and implement the FPGA-Accelerated DNN Inference Engine. The process is divided into Accelerator Generation Flow and System Integration Flow, culminating in the final SoC architecture.



3.2 System Architecture

The finalized hardware architecture is a heterogeneous System-on-Chip (SoC). The block diagram illustrates the connectivity between the central processor, the data fabric, and the custom accelerator.

The system is fundamentally divided into two major communication zones linked by the AXI Interconnect

3.2.1 The Processing System

- **MicroBlaze Soft Processor:**The central control unit responsible for managing the boot process, loading the CNN weights, initiating the accelerator, and handling communication with the host via UART.
- **Local Memory Bus (LMB):** Provides fast, low-latency access to Block RAM (BRAM) for the MicroBlaze's program and data.
- **AXI Peripherals:** TStandard I/O controllers such as AXI UART Lite for communication and the AXI Interrupt Controller (INTC) for managing signals from the accelerator.

3.2.2 The Custom Accelerator System

- **Custom CNN Accelerator IP:**The core computational engine generated by HLS. It contains the optimized parallel logic for convolutional operations.
  - **Control Interface:** Connected to the MicroBlaze via the AXI4-Lite protocol for register access (start/stop commands, reading status flags, passing parameters).
  - **Data Interface:** The main AXI interface (e.g., AXI4-Stream or AXI4-Full) is used for high-bandwidth data movement (input feature maps, weights, output results) to and from the main system memory (DDR or BRAM).
- **AXI Interconnect:** This central routing switch manages transactions, allowing the MicroBlaze master to communicate with all slave peripherals and the accelerator.

3.3 Working Principle:

The FPGA-Accelerated Deep Neural Network (DNN) Inference Engine operates on a Hardware/Software (HW/SW) co-design principle, where system control is managed by the software core and high-speed computation is delegated to the hardware accelerator. The complete execution occurs in four sequential phases connected through the AXI interconnect.

3.3.1 Phase 1: Software Configuration and Data Preparation

The MicroBlaze soft processor (software component) acts as the master controller responsible for system setup.

- **Data Placement:** The MicroBlaze loads neural network weights and the Input Feature Map (IFM) into Block RAM (BRAM), which acts as the accelerator's input buffer.
- **Accelerator Parameterization:** Configuration values such as IFM/OFM addresses, kernel size, and stride are written into Memory-Mapped I/O (MMIO) registers of the accelerator via AXI4-Lite.
- **Initiation:** The processor triggers the computation by writing to the `AP_START` register.

3.3.2 Phase 2: Hardware Computation and Data Movement

The custom CNN accelerator IP (hardware component), generated using HLS, performs all compute-intensive tasks.

- **Parallel Execution:** Unrolled loops and pipelining enable massively parallel Multiply-Accumulate (MAC) operations using FPGA DSP slices.
- **Dedicated Memory Access:** The accelerator streams IFM and weights directly from BRAM through its AXI data interface.
- **Result Staging:** The Output Feature Map (OFM) is written to a dedicated region in BRAM. The entire computation runs inside the FPGA fabric with minimal latency.

3.3.3 Phase 3: Hardware/Software Synchronization

Synchronization ensures that the software domain resumes only after the hardware completes processing.

- **Completion Signal:** When the output is fully written, the accelerator asserts a "Done" bit in its status register.
- **Interrupt Notification:** This flag triggers an Interrupt Request (IRQ) via the AXI Interrupt Controller (INTC), notifying the MicroBlaze without blocking.

3.3.4 Phase 4: Software Post-Processing and Output

The MicroBlaze performs the final steps to complete inference.

- **Verification and Post-Processing:** The processor checks the Done flag, clears the interrupt, and reads OFM data from BRAM. For multi-layer networks, this OFM becomes the IFM for the next layer.
- **System Output:** For the final layer, the result is transmitted off-chip via peripherals such as AXI UART Lite.

3.4 System Requirements

This section specifies the essential hardware and software components required to successfully realize the FPGA-Accelerated Deep Neural Network (DNN) Inference Engine using the Xilinx Vivado and Vitis toolchains.

3.4.1 Hardware System Requirements

The hardware environment is defined by the target FPGA and the system-level Intellectual Property (IP) cores that form the customized System-on-Chip (SoC) architecture.

Target Platform

The project is implemented on the following device, chosen for its balance of performance and resource availability suitable for deep learning inference.

Component	Specification	Purpose in Project
FPGA Device	Xilinx Artix-7 XC7A100T	Provides the Programmable Logic fabric (PL). BRAM and DSP Slices are used to store model weights and execute convolution operations.
System Clock	100MHz	Base frequency for the entire SoC. All timing constraints (WNS) must be met at this value.

MicroBlaze Soft Processor

MicroBlaze is a 32-bit RISC soft processor core implemented inside Xilinx FPGAs. It is synthesized from FPGA logic resources (LUTs, BRAMs, DSPs), making it configurable, lightweight, and ideal for embedded control in accelerator-based systems.

MicroBlaze executes software such as

- Accelerator control programs
- Data transfer routines
- Initialization and configuration processes

In this design, MicroBlaze acts as the system controller, configuring the CNN accelerator via AXI4-Lite and managing data movement through AXI4 and BRAM.

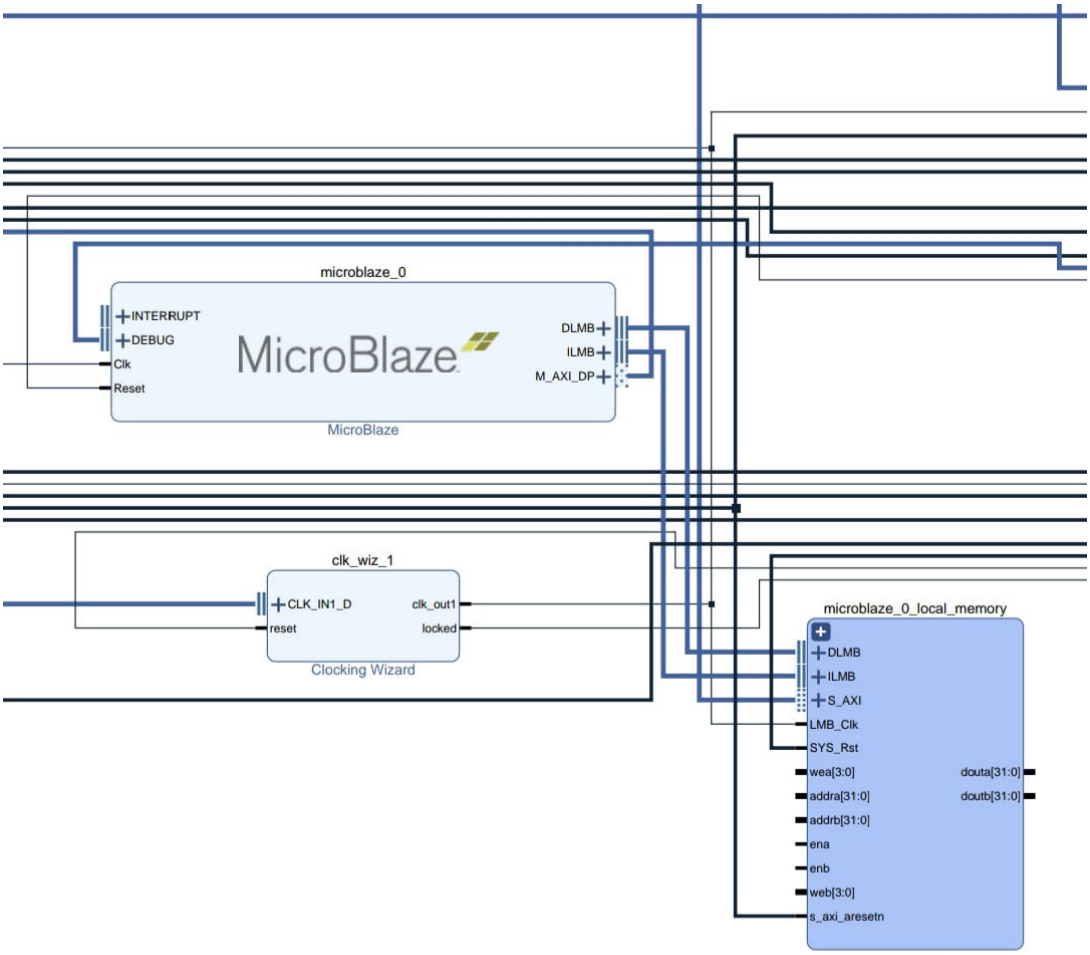


Figure 3.1: MicroBlaze IP

1. **MicroBlaze Architecture Overview**
   - 32-bit RISC Harvard architecture

- Separate instruction and data busses
- Optional hardware multiplier, divider, FPU
- Configurable cache sizes
- Customizable memory map
- Supports interrupts and exception handling

MicroBlaze is tightly integrated with AXI interfaces, making it ideal for SoC-style FPGA designs.

2. **Local Memory of MicroBlaze** MicroBlaze typically uses Block RAM (BRAM) inside the FPGA as local instruction and data memory.

   Local Memory Features:

   - On-chip low-latency memory
   - Connected through the Local Memory Bus (LMB) or AXI BRAM Controller
   - Used for:
     - Program code storage
     - Stack and heap
     - Temporary variables
     - Lookup tables

   In this design:

   - MicroBlaze uses BRAM to store its firmware.
   - The AXI BRAM Controller enables AXI-based access for both MicroBlaze and the CNN accelerator.

3. **MicroBlaze Clock (System Clock)**

   MicroBlaze requires a stable system clock for execution.

   Clock Features:

   - Typically generated from the Clocking Wizard
   - Common frequency: 50–150 MHz
   - All connected components must share or synchronize with this clock
   - Drives:
     - Instruction fetch pipeline
     - AXI transactions
     - Interrupt handling

  - Data bus operations

4. **Clock Wizard**

   The Clocking Wizard is an Xilinx IP that generates custom clock frequencies using PLL/MMCM primitives.

   Uses in this design:

   - Generates MicroBlaze system clock
   - Generates accelerator clock (if needed)
   - Provides stable, phase-aligned clock output

   Key Features:

   - Input: typically 100 MHz board clock
   - Output: configurable clock (50, 100, 150 MHz)
   - Provides reset and lock signals

5. **Overall Working of MicroBlaze in the System**

   - After reset, MicroBlaze loads and executes the BRAM software.
   - Configure the CNN accelerator using AXI4-Lite.
   - Manages data transfer to/from BRAM or DDR through AXI.
   - Polls for accelerator completion or handles interrupts.
   - Reads results and stores/prints the final output.

MicroBlaze provides a flexible and efficient soft processor inside the FPGA, supported by BRAM, AXI interfaces, and a configurable Clock Wizard. This combination enables high-speed execution and seamless control of the CNN accelerator in an SoC-style architecture.

Custom CNN Accelerator IP (Vitis HLS)

The `conv2d_accelerator` is a custom hardware IP core generated using Vitis HLS. Implements a 2D convolution operation in hardware using parallel MAC units, pipelining, and loop unrolling to achieve high throughput compared to software execution on MicroBlaze. This IP handles feature map inputs, filter weights, and biases, and produces the output feature map of the convolution layer.

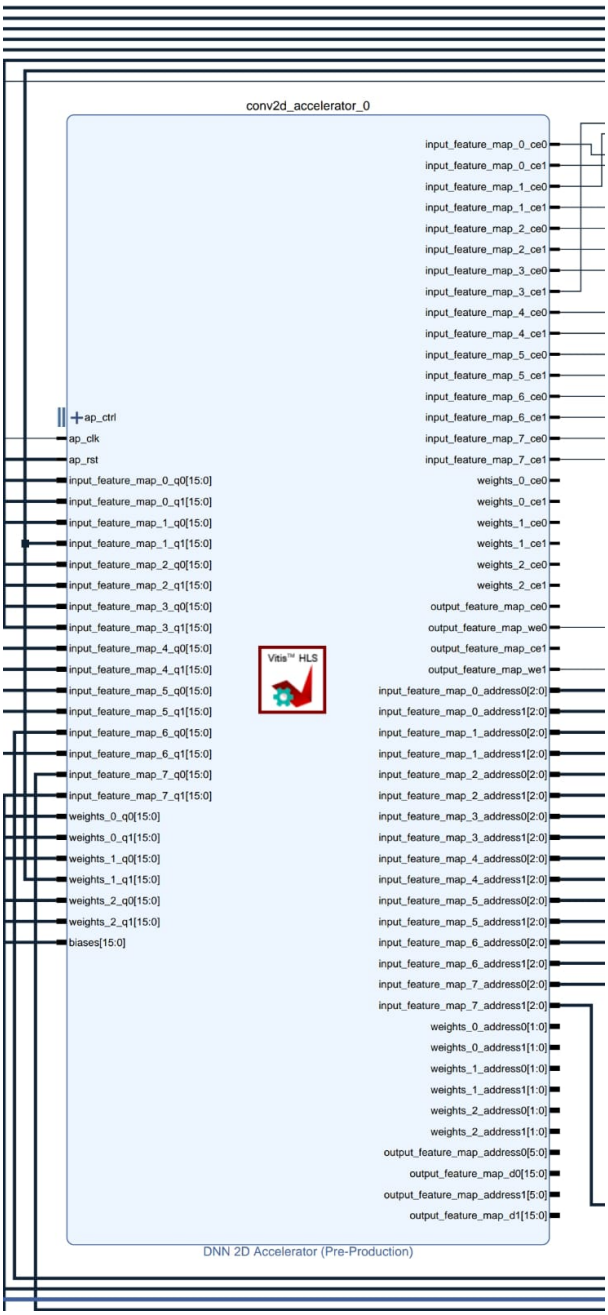


Figure 3.2: Custom CNN Accelerator

1. Purpose of the IP

- Performs 2D Convolution operation on hardware
- Accelerates deep learning workloads
- Reduces execution time compared to MicroBlaze software implementation
- Interfaces with memory and processor through BRAM ports

This IP is the core computational engine of your project.

2. Architecture Overview The accelerator internally uses:

- Local window buffers (for the 3×3 convolution)
- Parallel MAC (Multiply–Accumulate) units
- Pipelined loops (II=1 scheduling)
- Array partitioning & loop unrolling for parallel reads

These optimizations allow the accelerator to compute one output pixel per clock cycle under ideal conditions.

3. Ports and Interfaces Your IP is generated using BRAM interfaces (not AXI). So Vitis HLS created many address, data, and chip enable (ce) ports automatically for each memory.

3.1 Input Feature Map Ports Memory-mapped BRAM ports for the input image. Ports follow naming:

- `input_feature_map_0_q0`
- `input_feature_map_0_address0`
- `input_feature_map_0_ce0`

Each port corresponds to one BRAM interface that is used to store pixel data. the address ports select the pixel, q0 carries the data, and ce0 enables the read.

3.2 Weight Memory Ports Ports such as

- `weights_0_q0`, `weights_1_q1`, `weights_2_q0`
- `weights_0_address0`, etc.

These handle filter weights and support the parallel reading of kernel values 3×3.

3.3 Bias Port A simple 1-D input port:

- `biases[15:0]`

3.4 Output Feature Map Ports Ports to write the generated convolution result:

- `output_feature_map_0_d0`
- `output_feature_map_0_address0`
- `output_feature_map_0_we0`

d0 = data write, address0 = output pixel index, we0 = write enable.

4. Control Signals The HLS tool automatically generates a small control interface:

- `ap_ctrl`
- `ap_start` – MicroBlaze asserts this to start convolution
- `ap_done` – IP asserts when the computation finishes
- `ap_idle` – Indicates idle state
- `ap_ready` – Ready for a new operation

These allow MicroBlaze to control the accelerator like a peripheral.

5. Working Principle

1. MicroBlaze writes input feature maps, weights, and biases to BRAM.
2. MicroBlaze triggers the accelerator using `ap_start`.
3. The accelerator reads the input pixels using the address and ce0 ports.
4. Read 3×3 window → performs parallel MAC → accumulates the result.
5. Writes output pixel to output BRAM through write ports.
6. When the complete output map is computed, `ap_done` is asserted.
7. MicroBlaze reads the "output feature map" from BRAM.

This process ensures high-speed convolution without involving the processor in heavy computation.

This custom Vitis HLS IP:

- Implements 3×3 convolution using optimized hardware
- Interacts with BRAM through input/output ports
- Is fully controlled by MicroBlaze through `ap_ctrl` signals
- Forms the central deep learning accelerator in your FPGA system

It handles all the compute-intensive operations of CNN inference, enabling a complete hardware–software co-design.

Advanced eXtensible Interface (AXI)

The AXI (Advanced eXtensible Interface) protocol is part of the AMBA4 standard and serves as the main communication bus in the FPGA system. It enables high-speed, low-latency data transfer between the MicroBlaze processor, BRAM/DDR memory, and the custom CNN accelerator IP. AXI provides independent read and write channels, supports parallel transactions, and is widely used in SoC-based FPGA designs.

1. Types of AXI Used

AXI4

- High-performance memory-mapped interface
- Supports burst transfers
- Used for transferring:
  - Input feature maps
  - Weights
  - Output feature maps

AXI4-Lite

- Simplified low-bandwidth control interface
- No burst mode
- Used by MicroBlaze to configure the accelerator:
  - Start signal
  - Done/status signals
  - Parameter and address configuration

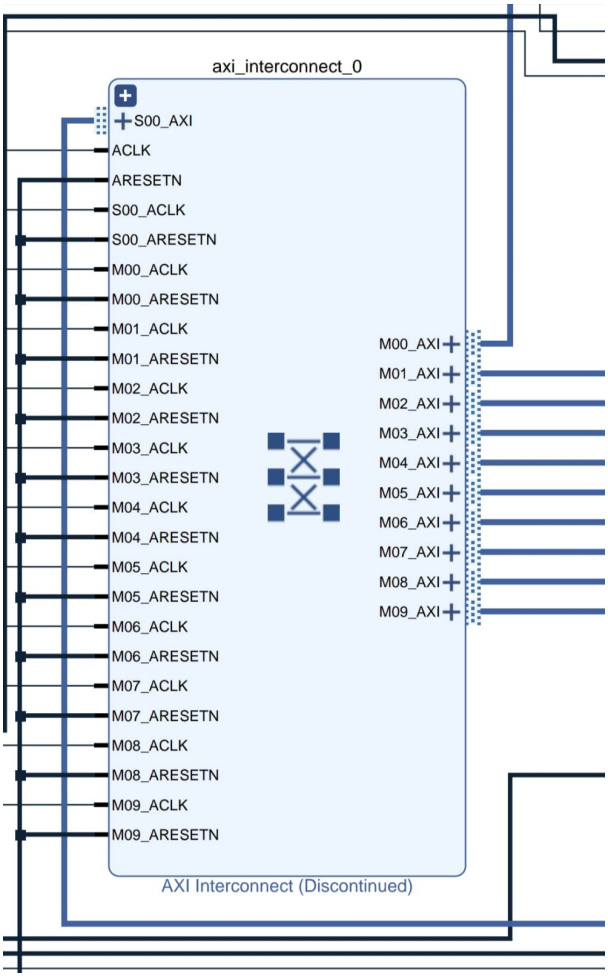


Figure 3.3: AXI Interconnect Diagram

AXI4-Stream

- Stream-based interface with no addressing
- Suitable for continuous dataflow
- Included for completeness in accelerator-based designs

2. AXI Channels and Ports AXI consists of five independent channels, each with its own set of signals.

Read Address Channel (AR)

- Signals: `ARADDR`, `ARVALID`, `ARREADY`

Read Data Channel (R)

- Signals: `RDATA`, `RVALID`, `RREADY`, `RLAST`

Write Address Channel (AW)

- Signals: `AWADDR`, `AWVALID`, `AWREADY`

Write Data Channel (W)

- Signals: `WDATA`, `WSTRB`, `WVALID`, `WREADY`, `WLAST`

Write Response Channel (B)

- Signals: `BRESP`, `BVALID`, `BREADY`

Key Point: AXI supports fully independent read and write operations, enabling high throughput.

3. Working of AXI in the System

1. **Configuration** MicroBlaze writes to the CNN accelerator's AXI4-Lite registers to send control signals such as `Start`, `Reset`, and operational parameters.

2. **Data Transfer** The accelerator reads Input feature maps and Convolution weights from BRAM/DDR via AXI4.

3. **Processing** The CNN accelerator performs convolution using internal parallel MAC units and pipelined dataflow for high-throughput execution.

4. **Result Storage** The accelerator writes the generated output feature maps back to BRAM/DDR through the AXI4 data interface.

5. **Completion** MicroBlaze reads the status register, acknowledges task completion, and retrieves the output data for further processing or transmission.

AXI provides a flexible, standardized, and high-performance communication interface that connects the MicroBlaze processor, memory, and the CNN accelerator IP. Its separation of read/write channels (AXI4), simple register-based control (AXI4-Lite) and fast data movement make it ideal for FPGA-based deep learning accelerators.

AXI BRAM Controller and Block RAM (BRAM)

The design uses multiple BRAM blocks and AXI BRAM Controllers to store the input feature maps, weights, biases, and output feature maps required by the CNN accelerator. Each BRAM block is connected directly to the corresponding memory port of the custom IP to allow fast, parallel access during convolution. The AXI BRAM Controllers serve as an interface between the MicroBlaze processor and the BRAMs, enabling the processor to load data into memory and read the results back after acceleration.

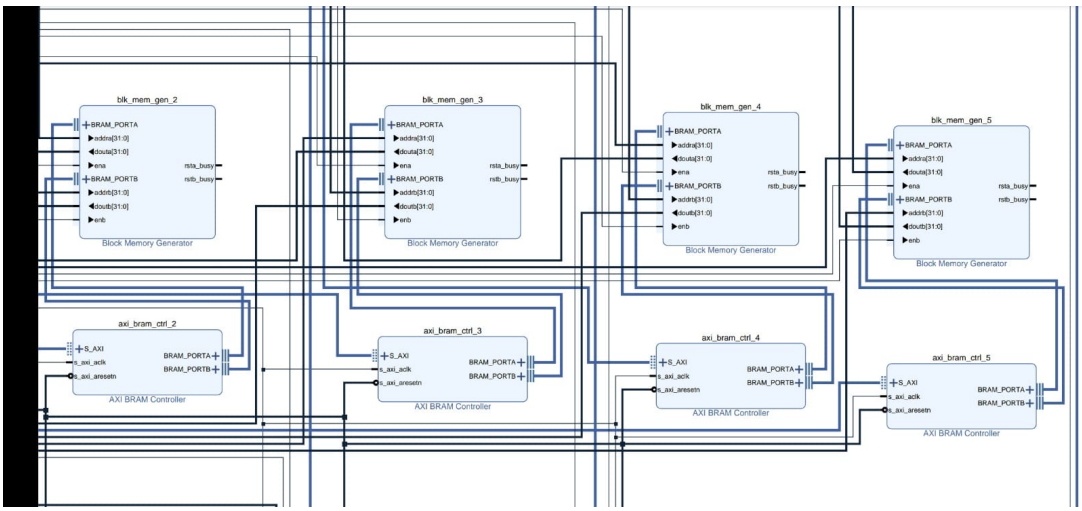


Figure 3.4: AXI BRAM Controller and Block RAM

1. Block RAM (BRAM) BRAM is the on-chip memory available inside the FPGA fabric. It is used here to store:

- Input feature map(s)
- Convolution weights
- Bias values
- Output feature map

Each BRAM instance is created using an IP of Block Memory Generator .

1. **Features**

    - Dual-port memory (PORT A and PORT B)
    - Independent read/write ports
    - Low-latency access
    - Fully synchronous operation

- Configurable width and depth

2. **Ports in the Block Diagram** Each BRAM block includes:

   - `addra`, `addrb` – address inputs for both ports
   - `douta`, `doutb` – data output ports
   - `ena`, `enb` – enable signals
   - Optional `we` (write enable) for writing data

This allows the CNN accelerator to read 3×3 window data in parallel, while MicroBlaze can access memory simultaneously if required.

2. AXI BRAM Controller The AXI BRAM Controller provides an AXI4-Lite slave interface to access MicroBlaze BRAM.

1. **Purpose**

   - Allows MicroBlaze to write input data (feature maps, weights, biases) into BRAM
   - Allows MicroBlaze to read output feature map after computation
   - Acts as a bridge between AXI bus and BRAM ports

2. **Ports**

   - `S_AXI` – AXI4-Lite interface used for memory-mapped access
   - `s_axi_aclk` – AXI clock
   - `s_axi_aresetn` – AXI reset
   - `BRAM_PORTA`, `BRAM_PORTB` – Connections to the Block RAM ports

The controller decodes AXI addresses and generates the correct BRAM enable, write enable, address, and data signals.

3. Why Multiple BRAMs Are Used Your custom CNN accelerator IP (from Vitis HLS) has many memory ports for:

- Input feature map channels
- Weight matrices
- Bias values
- Output feature map

To support parallel data access, each of these memory interfaces requires a separate BRAM instance.

1. **Advantages of Multiple BRAMs**

   - All memory reads for a 3×3 window can occur in the same clock cycle
   - Weight reads can occur simultaneously
   - Output writes do not stall the input reads
   - Dramatically increases accelerator throughput

This is essential for hardware convolution, where parallel access is the key performance advantage.

4. Working Flow

1. MicroBlaze writes data to BRAM through the AXI BRAM Controller using AXI4-Lite.
2. CNN Accelerator reads BRAM contents directly through dedicated BRAM ports (`q0`, `address0`, `ce0`).
3. During convolution:

   - Input pixels are fetched in parallel
   - The weights are read simultaneously
   - The output is written back into the output BRAM

4. After the computation, MicroBlaze reads the results from the output BRAM for display or verification.

The combination of AXI BRAM Controllers and multiple Block Memory Generator instances creates a high-speed memory subsystem that supports the parallel data access requirements of the CNN accelerator. The BRAM blocks provide low-latency storage, while the AXI BRAM Controllers enable seamless integration with the MicroBlaze processor through the AXI4-Lite interface.

AXI UART Lite

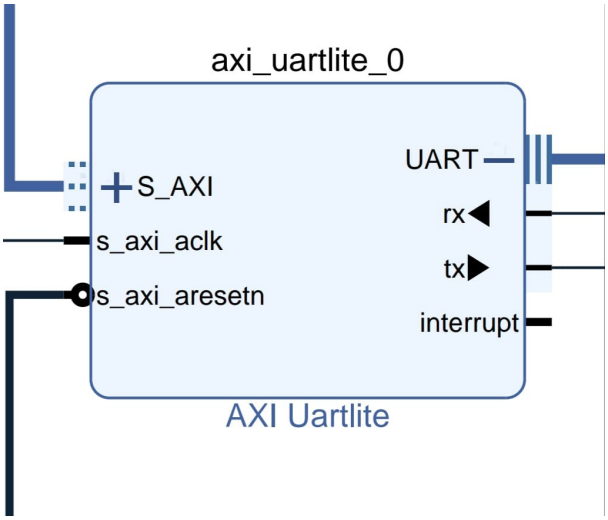


Figure 3.5: AXI UART Lite IP

The AXI UART Lite is a lightweight Universal Asynchronous Receiver/Transmitter (UART) module designed for simple serial communication between the FPGA system and a host computer. It is connected to the MicroBlaze processor through an AXI4-Lite interface, enabling software running on the processor to send and receive data via a serial terminal. UART Lite is ideal for printing debug messages, viewing output feature maps, or monitoring accelerator results.

1. **Purpose in the System**

   Provides a serial communication channel between FPGA and PC.

   Used for:

   - Printing program output
   - Debugging
   - Displaying convolution results
   - Sending configuration or commands

2. **AXI Interface**

   AXI4-Lite Slave Port (S_AXI)

   Used by MicroBlaze to access UART registers:

   - Transmit (TX) register
   - Receive (RX) register
   - Status register
   - Control register

Clock and Reset

- `s_axi_aclk` – AXI clock
- `s_axi_aresetn` – Active-low reset

These signals synchronize the UART with the MicroBlaze system.

3. **UART Ports**

   rx (Receive) Receives serial data from the PC to the FPGA. Used when user inputs or commands are needed.

   tx (Transmit) Sends serial data from FPGA to PC. Used to display program output (e.g., convolution results).

   interrupt Can trigger an interrupt in MicroBlaze when:

   - New data is received
   - Transmission is complete.

   (Interrupt use is optional depending on the software design.)

4. **Working Principle**

   (a) MicroBlaze writes a byte to the UART TX register → transmitted to the PC via `tx`.

   (b) Incoming serial data are placed into the RX register → MicroBlaze reads it when required.

   (c) UART Lite operates at a configured baud rate (typically 9600 or 115200 baud).

   (d) The PC serial terminal displays all transmitted messages for debugging or monitoring.

5. **Summary**

   The AXI UART Lite module provides a simple and efficient serial communication interface within the system. It enables MicroBlaze to interact with external devices, print debug information, and display accelerator outputs, making it an essential peripheral for monitoring and controlling the CNN hardware accelerator system.

AXI Interrupt Controller (INTC)

- **Role:** Aggregates IRQ signals from peripherals. Most importantly, receives the "Done" interrupt from the accelerator and forwards it to MicroBlaze.
- **Interface:** AXI4-Lite Slave for configuration, plus a direct interrupt line to MicroBlaze.

3.4.2 Software System Requirements

Hardware Development Suite

- **Xilinx Vivado Design Suite:** Used for synthesis, implementation, block design, and generating the hardware platform (XSA file).

High-Level Synthesis Tool

- **Xilinx Vitis HLS:** Converts C/C++ functions to RTL and generates the custom CNN accelerator.

Software Development Environment

- **Xilinx Vitis Unified Software Platform:** Used to create and debug MicroBlaze C/C++ application. Generate BSP (Board Support Package) with API drivers.

MicroBlaze Firmware

- **Language:** C/C++
- **Role:** Implements the full multi-phase inference procedure described in Section 3.5, including:
  - The AXI register writes
  - Data transfers
  - Interrupt handling
  - Layer-wise processing

Chapter 4

Implementation Process

The complete implementation of the CNN Hardware Accelerator followed a structured hardware–design flow using Vitis HLS and Vivado Design Suite. The workflow includes C-level modeling, simulation, HLS synthesis, IP export, subsystem construction in Vivado, and full implementation up to post–route and packaging.

4.1 C-Level Model Creation

A high-level C/C++ model of the 2D convolution operation was created. The model implements the following steps:

- Reading the input feature map
- Performing 3×3 kernel multiplications
- MAC (Multiply–Accumulate) computation
- Bias addition

This function serves as the reference computational model for the hardware accelerator.

4.2 C/C++ Simulation (CSIM)

C-level simulation (CSIM) was performed in Vitis HLS to ensure functional correctness of the model.

- Input arrays, weights, and biases were initialized.
- The output was validated against a golden reference.
- The simulation confirmed the algorithm correctness before synthesis.

4.3 HLS Synthesis and Optimization

The C model was synthesized into RTL using Vitis HLS. Several hardware-centric optimizations were applied:

- Loop pipelining with Initial Interval (II) = 1
- Loop unrolling
- Array partitioning for parallel memory access
- BRAM inference for mapping arrays to on-chip memory

The synthesis step produced RTL (Verilog/VHDL), latency estimates, performance analysis, and resource utilization reports.

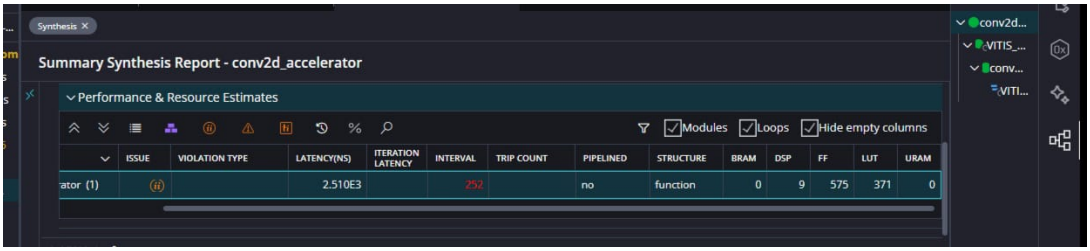


Figure 4.1: Initial Synsthesis

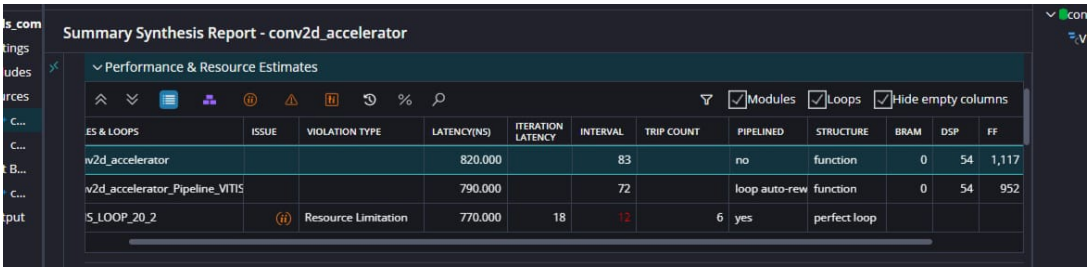


Figure 4.2: Optimization - 1

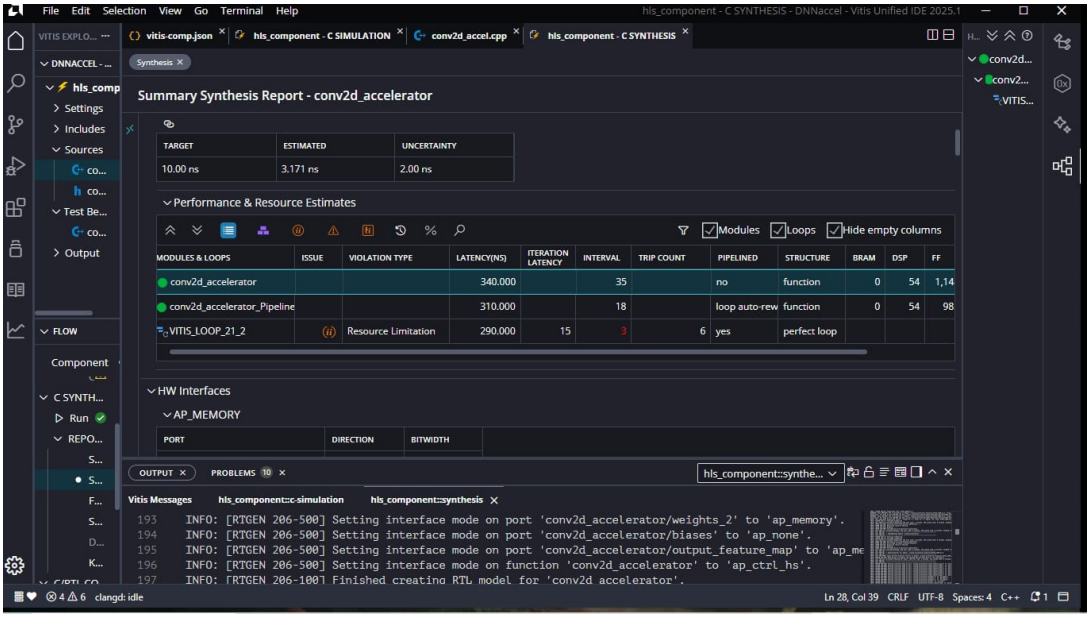


Figure 4.3: Final Optimization

HLS optimization significantly improved the performance of the convolution accelerator. The initial design showed a latency of 2510 ns. After applying loop pipelining, loop unrolling, and memory optimization directives, the latency was reduced to 340 ns in the final design. The initiation interval (II) was also improved from 12 to 3 in critical loops. These optimizations increased resource usage, especially DSP and flip-flop utilization, but achieved substantial improvements in throughput and computational efficiency

4.4 Export RTL IP Core

After verification, the CNN accelerator was exported as a reusable custom IP. The exported IP package included:

- RTL source files
- IP metadata and configuration files
- AXI-Lite control interface (ap_start, ap_done)
- BRAM-style memory interfaces for input, weights, and output

Custom IP was added to the Vivado IP catalog for further integration of the system.

4.5 Create Block Diagram in Vivado

A new Vivado block design was created to integrate the accelerator into a complete hardware subsystem. The major components added include:

- MicroBlaze soft processor
- AXI Interconnect
- AXI BRAM Controllers
- Multiple BRAM blocks for input, weights, and output
- AXI UART Lite for serial communication
- Clocking Wizard
- Processor System Reset

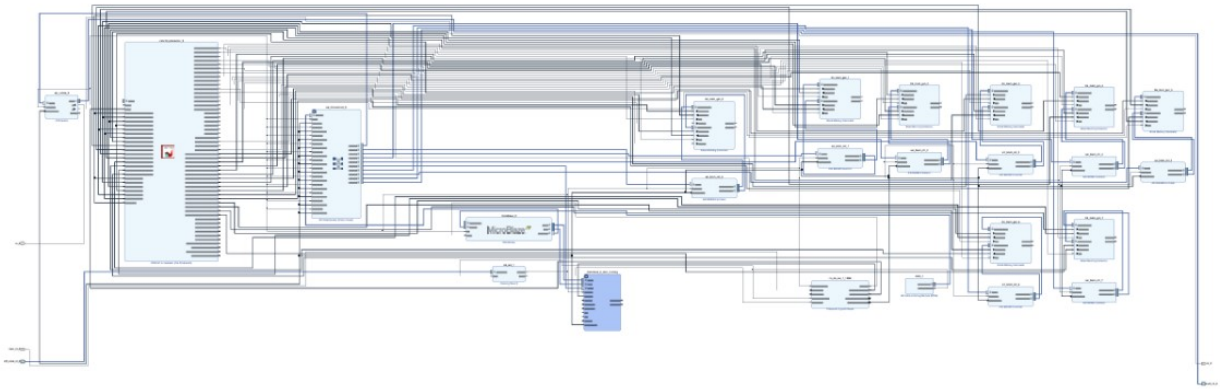


Figure 4.4: Block Diagram

The custom CNN accelerator IP was connected to BRAMs, AXI interfaces, and system clocks.

4.6 Peripheral Configuration

Each component of the subsystem was configured to meet the design specifications.

- **MicroBlaze:** clock configuration, local memory, AXI interfaces
- **BRAM Generators:** memory depth, width, and dual–port architecture
- **AXI BRAM Controllers:** control and data interface mapping
- **UART Lite:** baud rate and AXI-Lite register configuration
- **CNN Accelerator IP:** BRAM and AXI-Lite control port connections

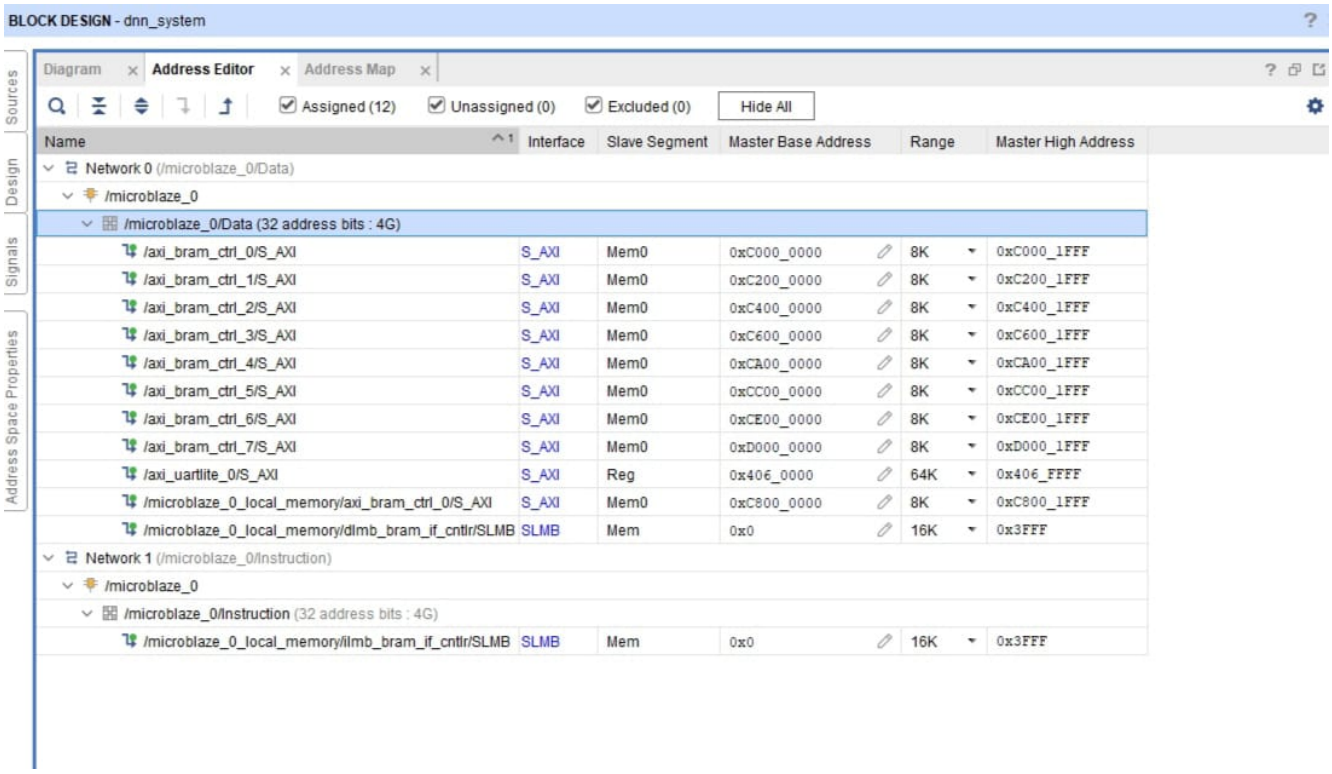


Figure 4.5: Address Editor

Vivado Address Editor was used to assign non-overlapping AXI address ranges for all peripherals.

4.7 Generate HDL Wrapper

Vivado generated a top-level HDL wrapper that instantiates:

- All IP blocks in the design
- AXI interconnect wiring
- Clock and reset distribution networks

```
`timescale 1 ps / 1 ps

module dnn_system_wrapper
   (diff_clock_rtl_0_clk_n,
    diff_clock_rtl_0_clk_p,
    reset_rtl_0,
    rx_0,
    tx_0);
  input diff_clock_rtl_0_clk_n;
  input diff_clock_rtl_0_clk_p;
  input reset_rtl_0;
  input rx_0;
  output tx_0;

  wire diff_clock_rtl_0_clk_n;
  wire diff_clock_rtl_0_clk_p;
  wire reset_rtl_0;
  wire rx_0;
  wire tx_0;

  dnn_system dnn_system_i
       (.diff_clock_rtl_0_clk_n(diff_clock_rtl_0_clk_n),
        .diff_clock_rtl_0_clk_p(diff_clock_rtl_0_clk_p),
        .reset_rtl_0(reset_rtl_0),
        .rx_0(rx_0),
        .tx_0(tx_0));
endmodule
```

This wrapper serves as the top module for synthesis and implementation.

4.8 Implementation (Synthesis → Placement → Routing)

Vivado performed the full FPGA implementation flow:

- **Synthesis:** RTL translated into a gate-level netlist
- **Placement:** Logic mapped to FPGA slices, DSPs, and BRAMs
- **Routing:** Interconnections completed across the FPGA fabric

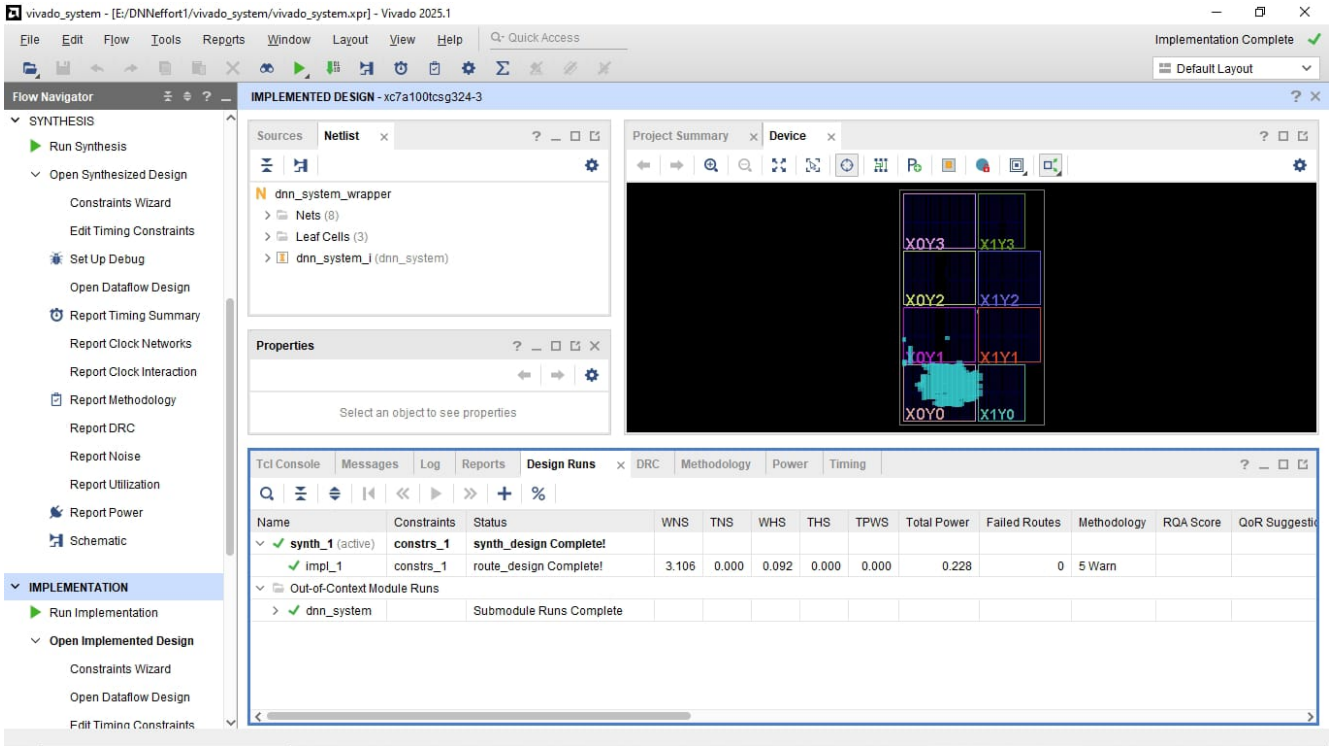


Figure 4.6: Implementation

Timing constraints were checked, and routing ensured correct clock-domain alignment and signal timing.

4.9 Post-Route Analysis

After routing, the final analysis included:

- Static Timing Analysis (setup and hold constraints)
- Resource utilization summary (LUT, FF, BRAM, DSP usage)
- Clocking and reset network validation

- Floor Planning and congestion analysis

This ensured that the design met the timing and physical requirements.

4.10 Packaging

Following successful implementation, the final output included the following

- Fully routed design
- FPGA bitstream file (.bit) generated through *Generate Bitstream*
- Design packaged for export *(software integration in Vitis not performed)*

This concludes the hardware implementation cycle for the CNN accelerator system up to the packaging stage.

Chapter 5

Results

This section presents the implementation and performance results of the proposed CNN hardware accelerator designed using Vitis HLS and the Vivado Design Suite. The design was successfully synthesized, placed, routed and verified for timing and power performance on the target FPGA.

5.1 Floorplanning Results

Floorplanning was performed to optimally assign logic resources to physical regions of the FPGA. The major hardware components such as the Custom CNN Accelerator IP, MicroBlaze processor, AXI Interconnect, and BRAM blocks were distributed throught the FPGA fabric.

The floorplan report confirms the following

- Logic cells (LUTs and Flip-Flops) were placed efficiently
- The DSP slices used for the multiply–accumulate operations were concentrated in optimal locations
- BRAM blocks were properly aligned near memory controller instances

This ensured reduced routing congestion and improved timing performance.

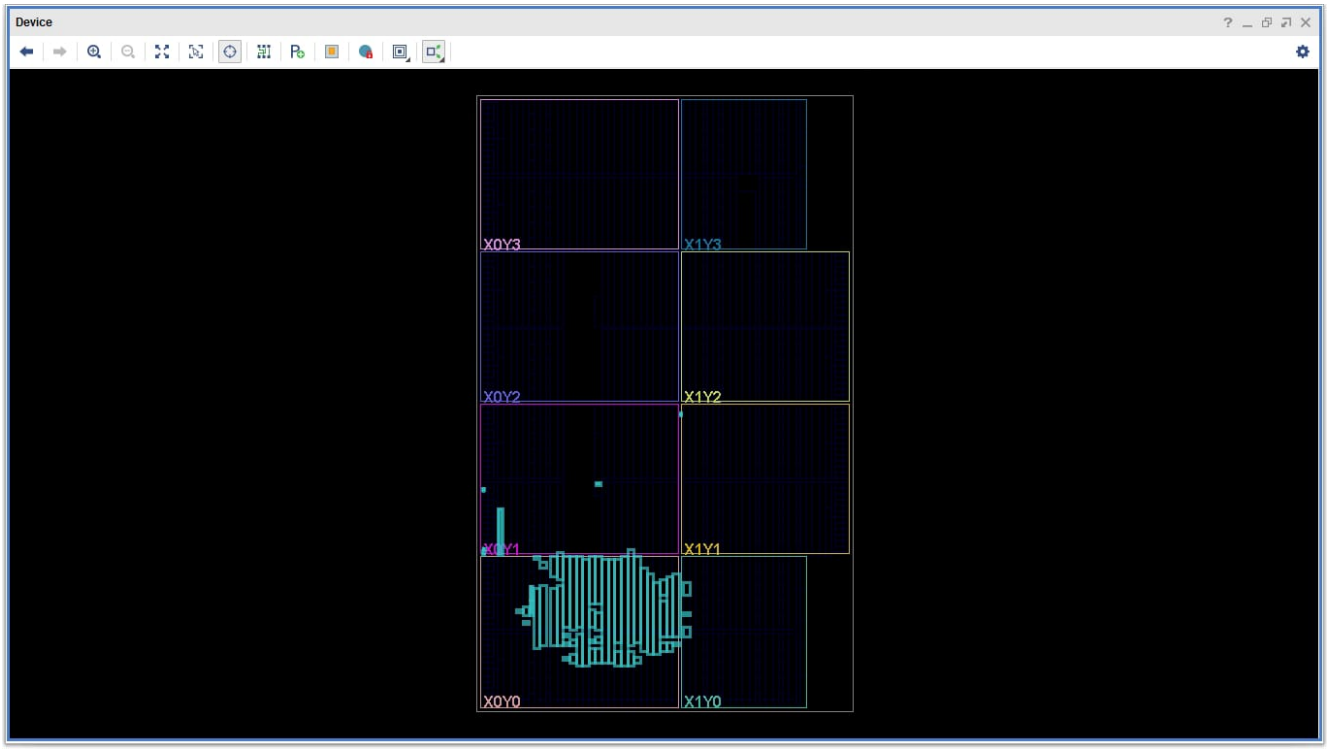


Figure 5.1: Floor Planning a

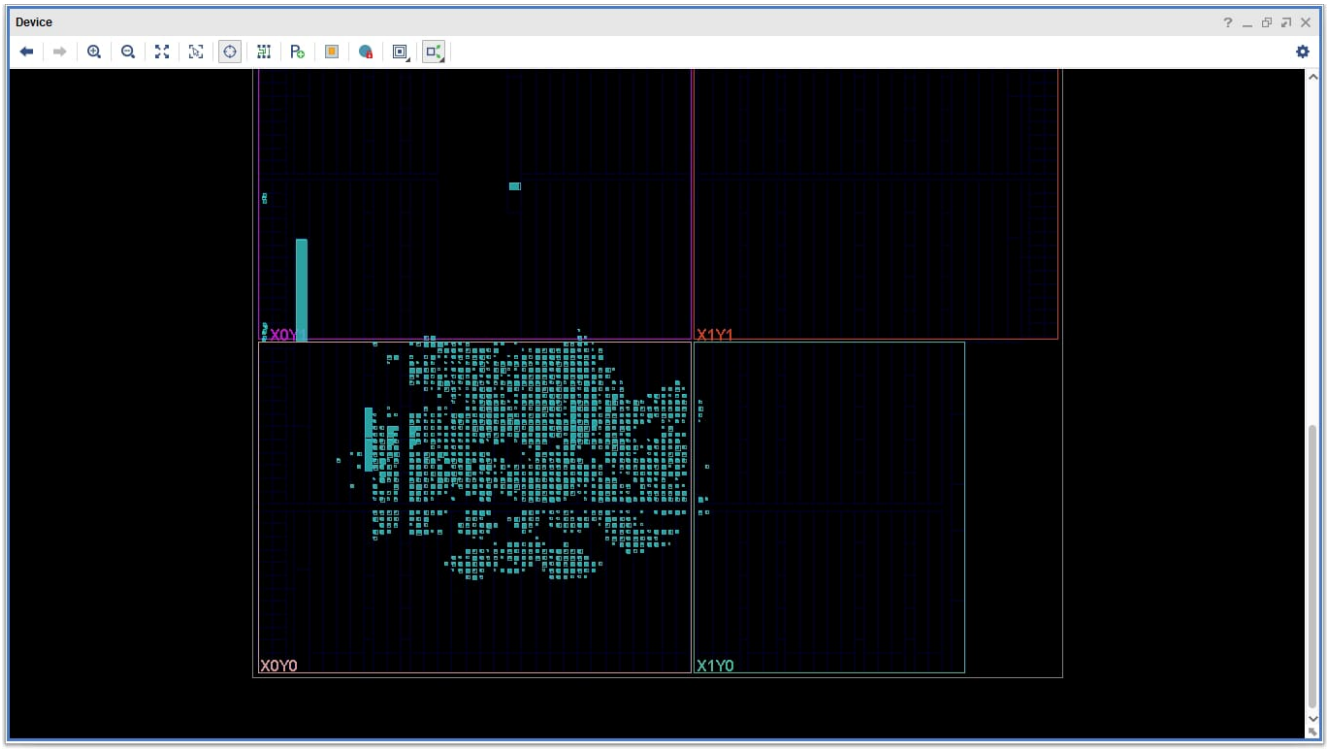
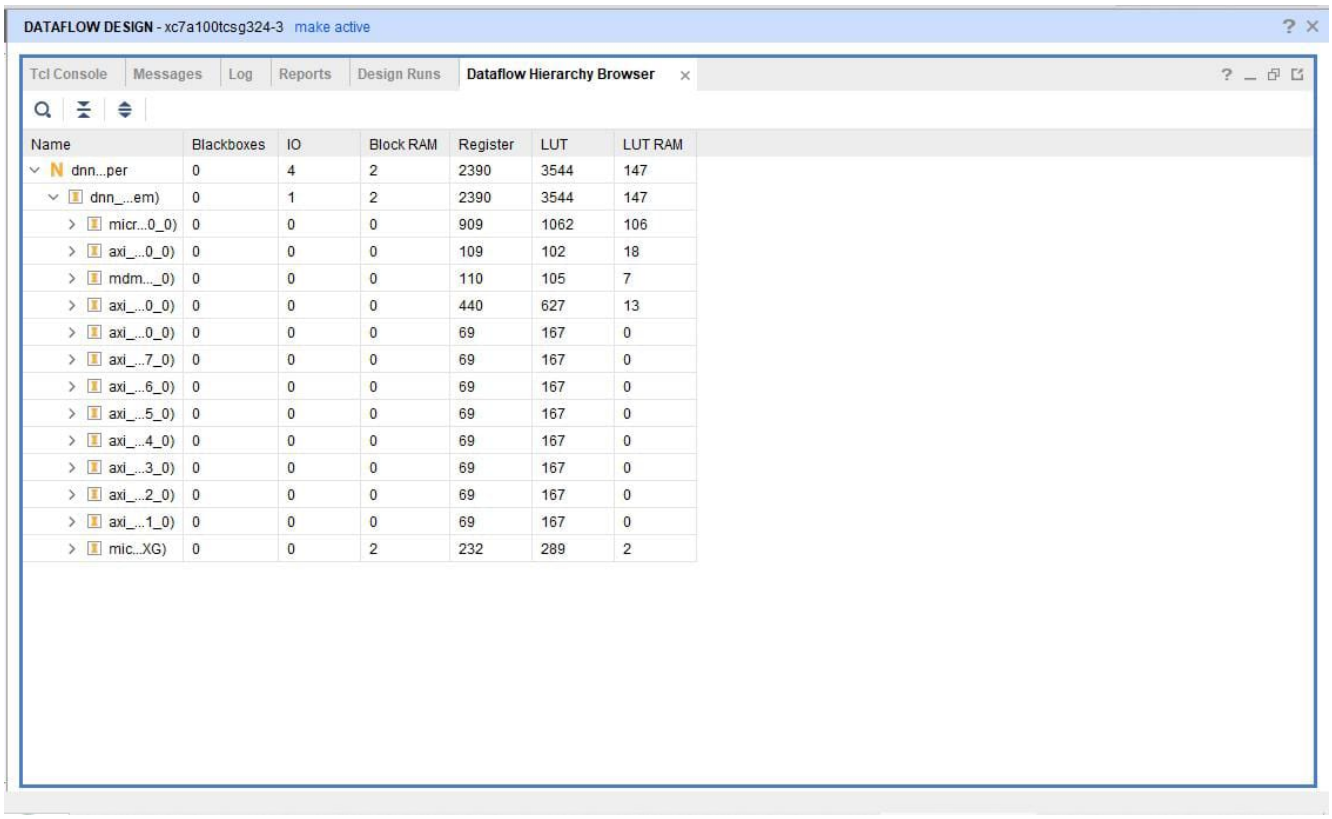


Figure 5.2: Floor Planning b

5.2 Resource Utilization Results

After synthesis and implementation of the CNN hardware accelerator, the utilization of FPGA resources was analyzed using the Vivado Design Suite. The design shows efficient usage of available hardware resources such as Lookup Tables (LUTs), Flip-Flops (FFs), DSP slices, and Block RAM (BRAM). The multiplier–accumulate operations of the convolution engine were mainly mapped onto DSP blocks, helping to achieve high computational efficiency with minimal logic overhead.

The overall LUT and Flip-Flop utilization remained low, while DSP usage was moderate due to the intensive arithmetic operations involved in convolution. Block RAM and Ultra RAM usage was minimal since most data buffering was handled through external and on-chip memories. These results confirm that the proposed architecture is area-efficient and scalable for larger CNN models.



Name	Blackboxes	IO	Block RAM	Register	LUT	LUT RAM
dnn...per	0	4	2	2390	3544	147
dnn_...em)	0	1	2	2390	3544	147
micr...0_0)	0	0	0	909	1062	106
axi_...0_0)	0	0	0	109	102	18
mdm..._0)	0	0	0	110	105	7
axi_...0_0)	0	0	0	440	627	13
axi_...0_0)	0	0	0	69	167	0
axi_...7_0)	0	0	0	69	167	0
axi_...6_0)	0	0	0	69	167	0
axi_...5_0)	0	0	0	69	167	0
axi_...4_0)	0	0	0	69	167	0
axi_...3_0)	0	0	0	69	167	0
axi_...2_0)	0	0	0	69	167	0
axi_...1_0)	0	0	0	69	167	0
mic...XG)	0	0	2	232	289	2

Figure 5.3: Resource Utilization Report

5.3 Routing Results

After placement, the routing process successfully connected all logic blocks using FPGA routing resources. The routing report indicated that:

- All critical signal paths were successfully routed
- No routing conflicts or congestion were reported
- Long interconnect delays were minimized through optimized placement

The successful routing confirms the physical feasibility of the proposed architecture.

```
Phase 8 Route finalize
Router Utilization Summary Global Vertical Routing Utilization = 0.569787 %
Global Horizontal Routing Utilization = 0.848323 % Routable Net Status*
Does not include unroutable nets such as driverless and loadless.
Run report_route_status for detailed report.
Number of Failed Nets = 0
(Failed Nets is the sum of unrouted and partially routed nets)
Number of Unrouted Nets = 0
Number of Partially Routed Nets = 0
Number of Node Overlaps = 0
Phase 8 Route finalize | Checksum:  29695f10c
Time (s):  cpu = 00:01:31 ; elapsed = 00:02:48 .  Memory (MB): peak = 2147.258
; gain = 135.902
Phase 9 Verifying routed nets
Verification completed successfully
```

Figure 5.4: Routing Console Output

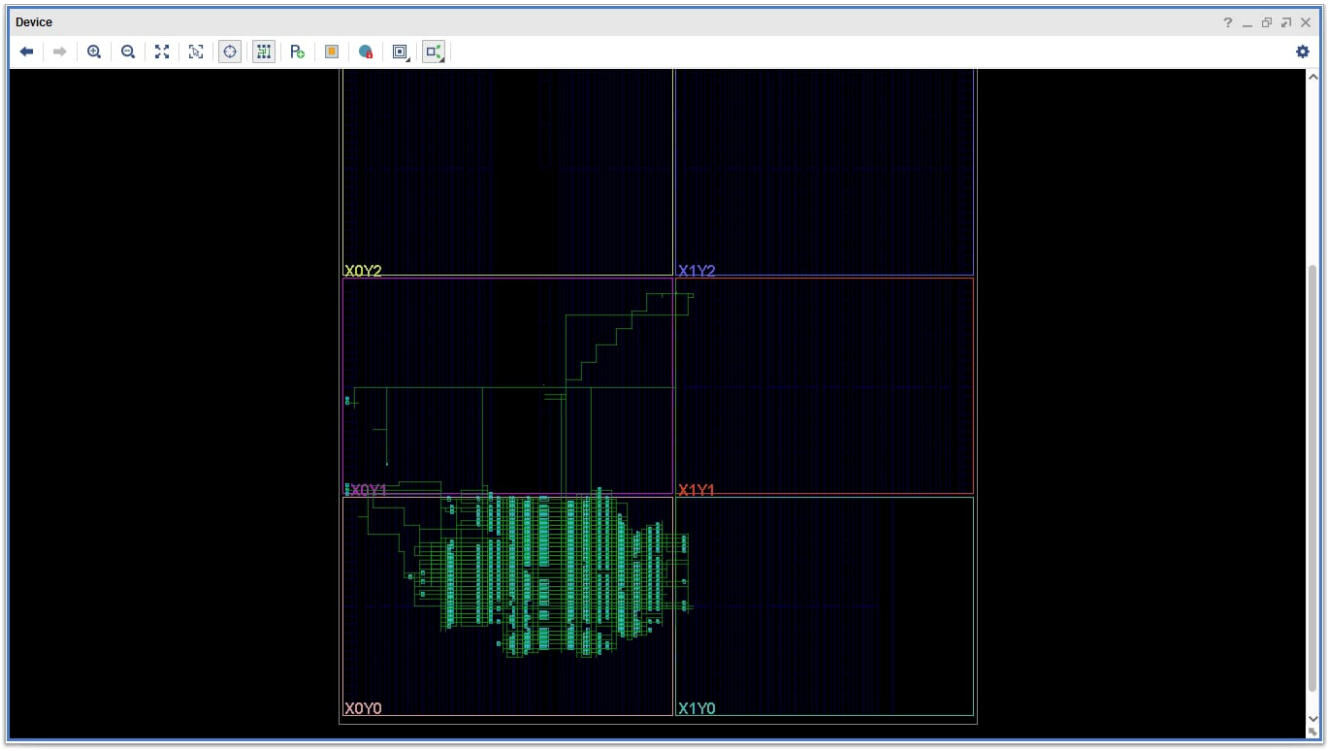


Figure 5.5: Routing a

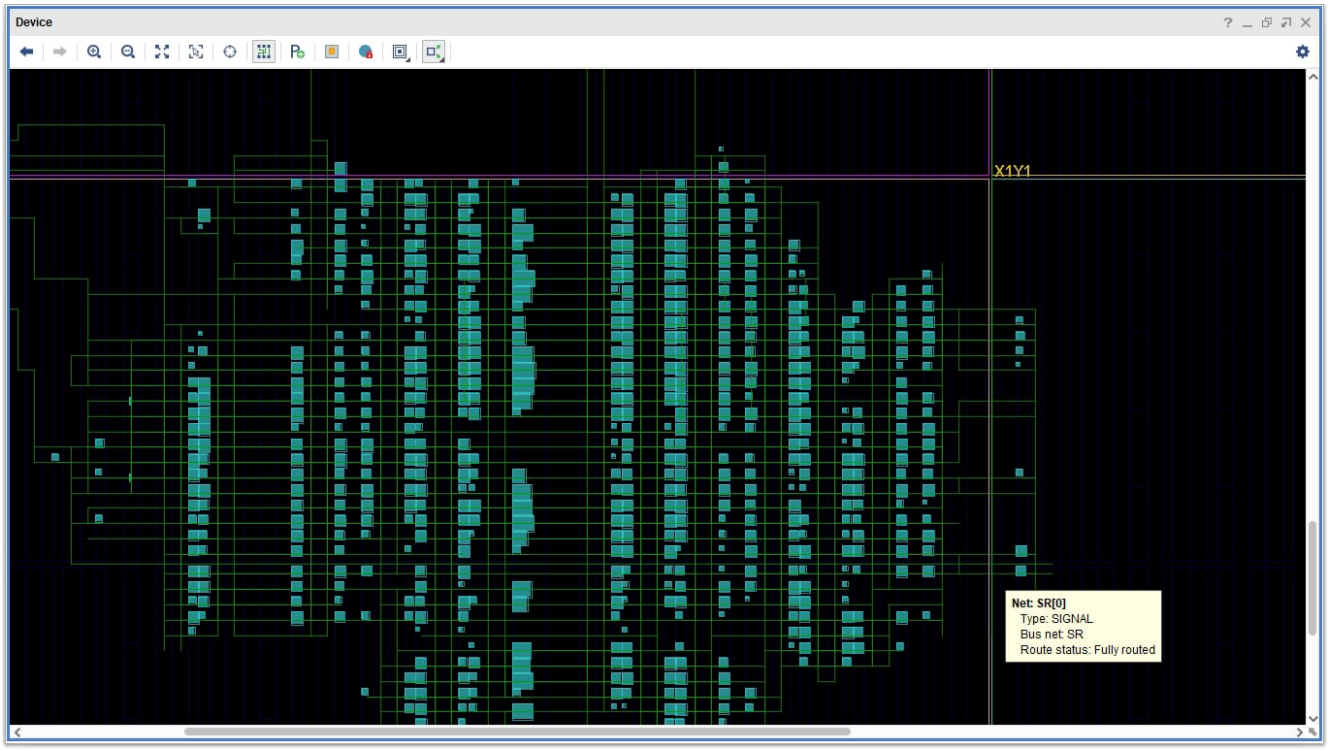


Figure 5.6: Routing b

5.4 Timing Analysis Results

Static Timing Analysis (STA) was performed after routing. The design met the target timing constraints.

Table 5.1: Timing Analysis Results

Parameter	Value
Target Clock Period	10.0 ns
Worst Negative Slack (WNS)	Positive (Timing Met)
Setup Violations	None
Hold Violations	None

The estimated critical path delay satisfied the clock frequency requirement, proving that the design is timing-safe for real hardware deployment.

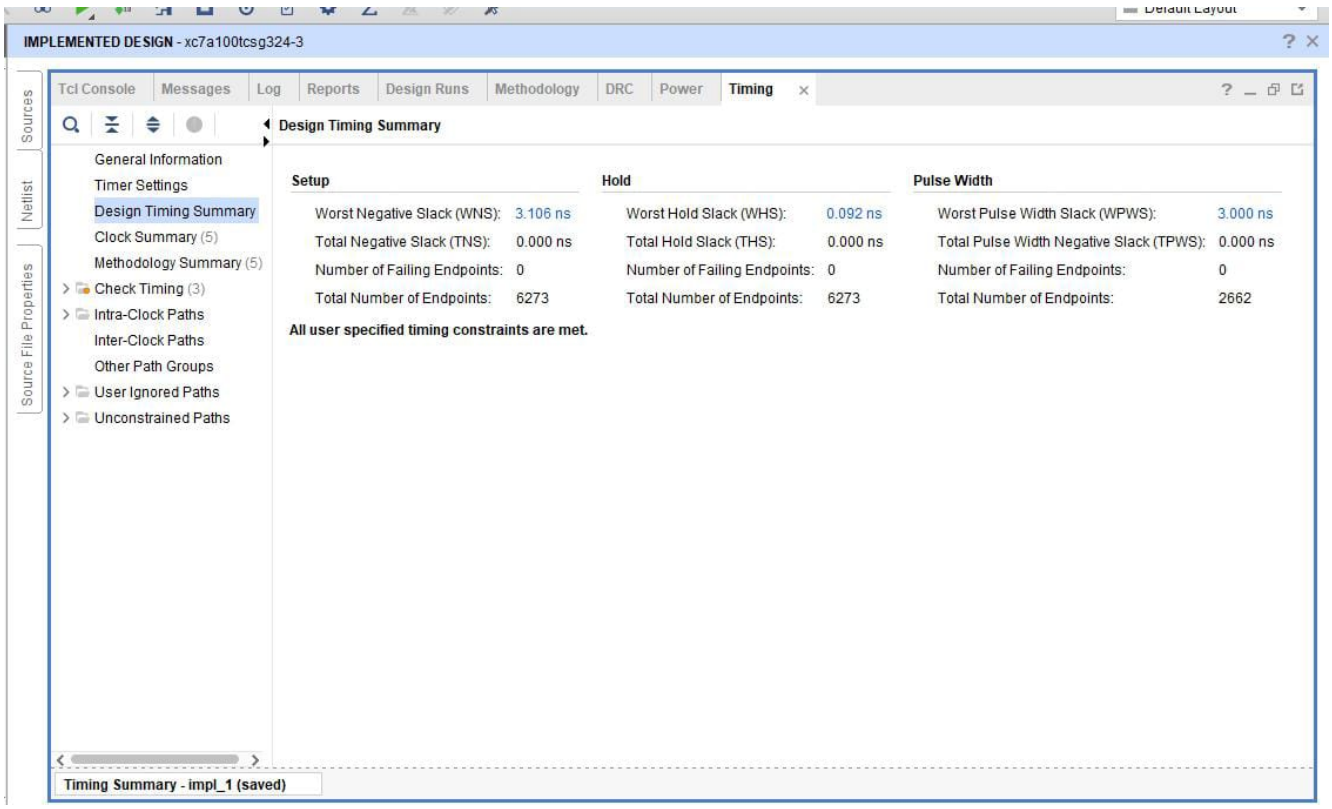


Figure 5.7: Timing Reports

5.5 Power Analysis Results

Power estimation was carried out using Vivado power analysis tools after routing. Both static and dynamic power components were evaluated.

Table 5.2: Power Analysis Results

Power Component	Observed Result
Static Power	0.092 W (low)
Dynamic Power	0.136 W (moderate)
Total Power	0.228 W (within FPGA limits)

The inclusion of BRAM and DSP blocks helped to reduce logic switching activity, resulting in improved power efficiency.

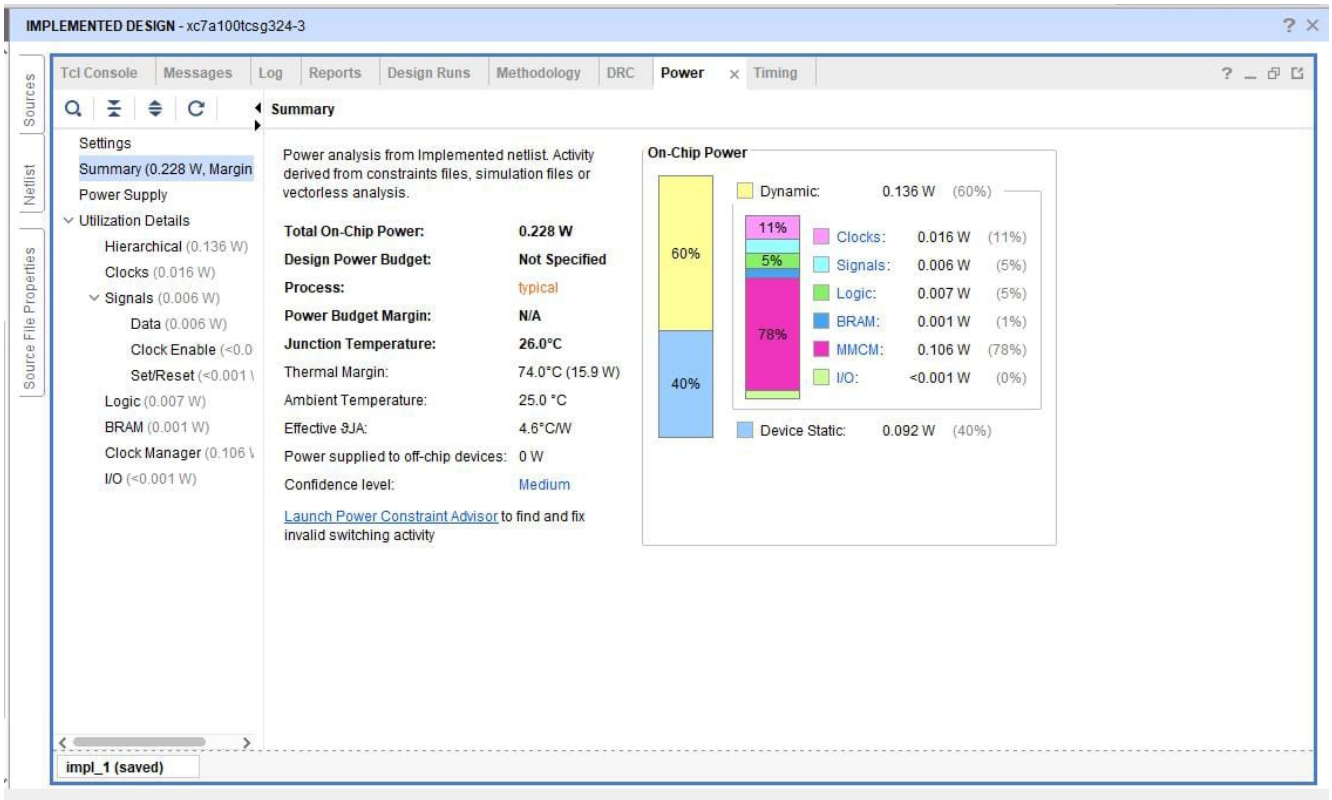


Figure 5.8: Power Report

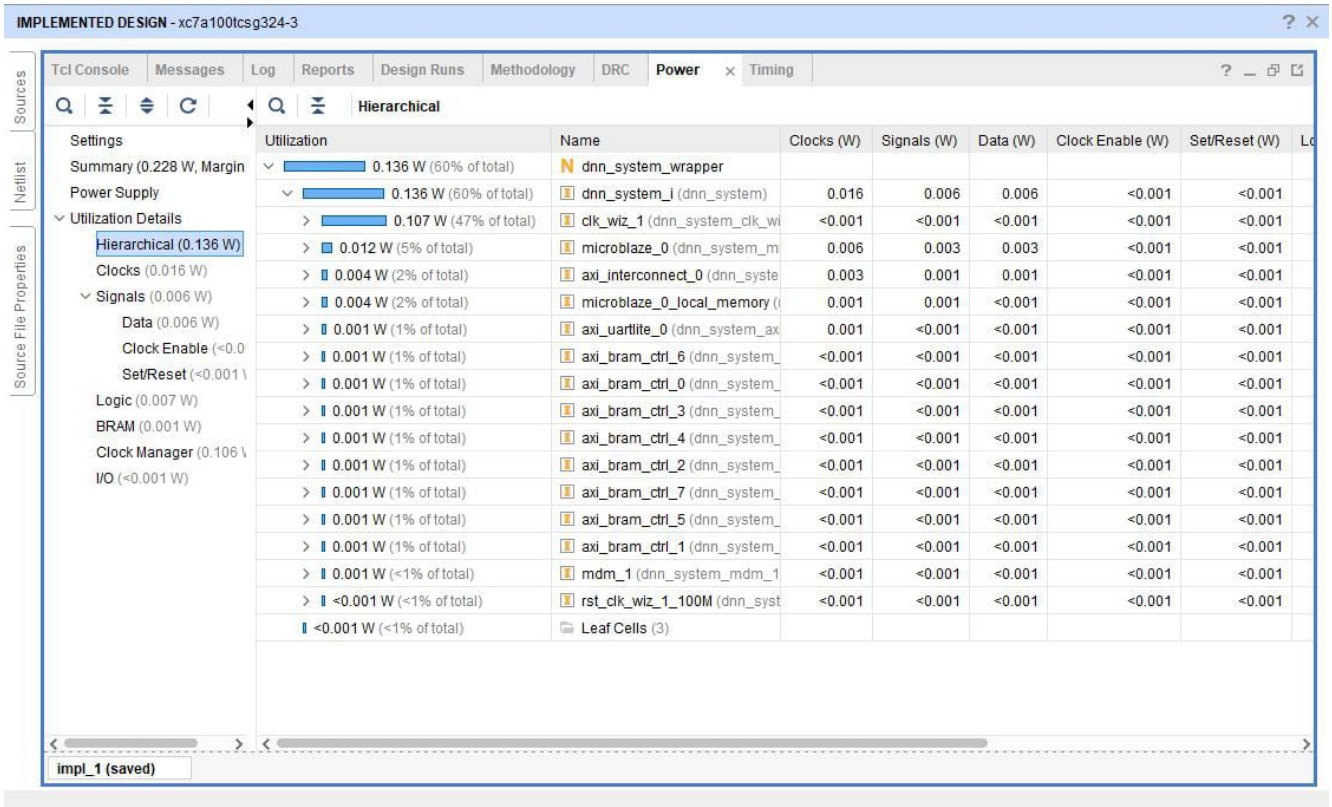


Figure 5.9: Power Utilization

5.6 RTL Schematic Results

The RTL schematic was generated after successful synthesis and implementation. The schematic shows the hierarchical connection between:

- MicroBlaze Processor
- AXI Interconnect
- Custom CNN Accelerator IP
- BRAM Controllers
- Memory Blocks
- UART Interface

The schematic verified the correct structural connectivity of all functional modules in the design.

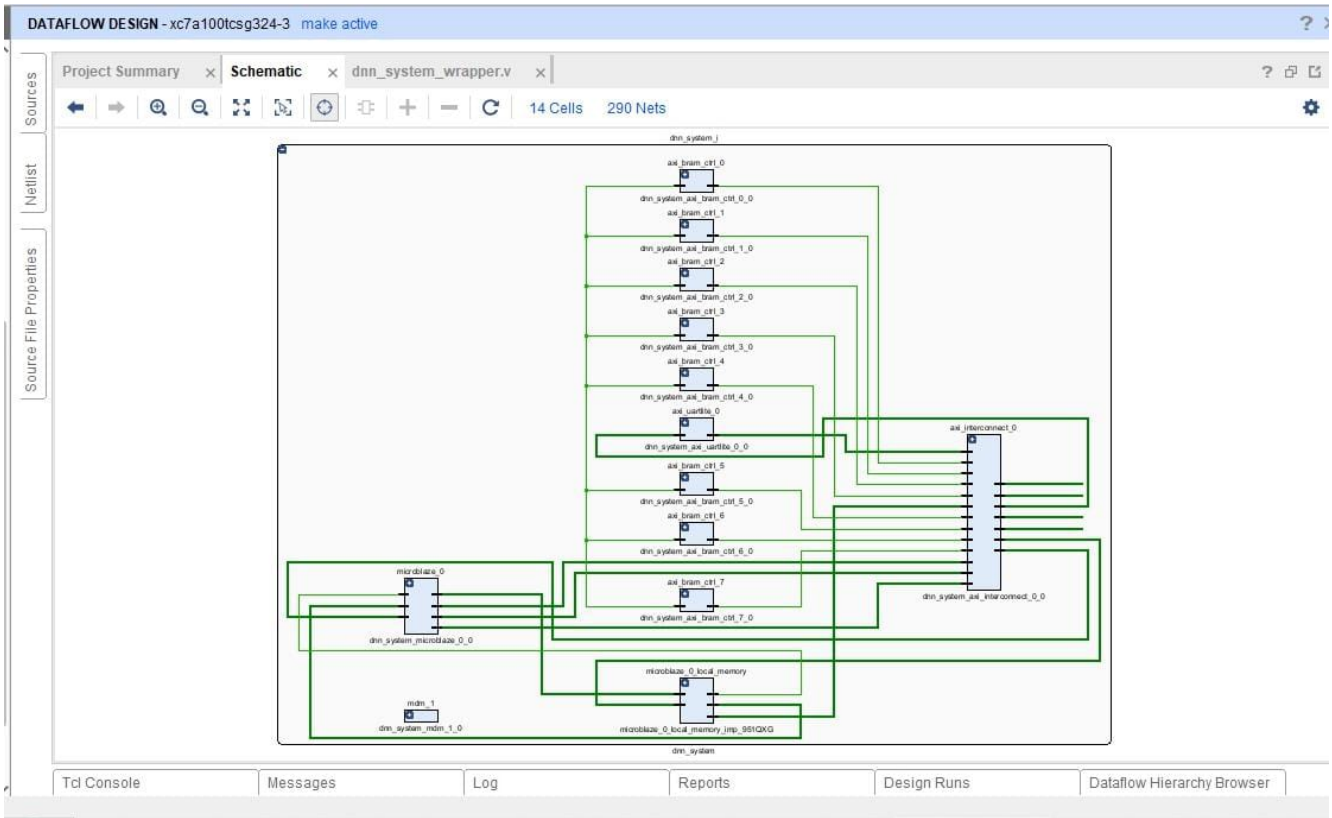


Figure 5.10: RTL Schematic

5.7 IP Packaging Results

The final step of the design flow was packaging the synthesized CNN accelerator as a reusable custom IP core. The IP was successfully packaged using the Vivado IP Packager, generating a reusable and parameterized IP block.

This packaged IP can be easily integrated into future designs using the Vivado IP Catalog, enabling rapid system-level design reuse.

The packaging report confirms successful generation of metadata files, HDL wrappers, and interface definitions for the custom CNN accelerator IP.

Figure 5.11: IP Package

5.8 Overall Implementation Summary

Table 5.3: Overall Implementation Summary

Design Parameter	Result
Design Status	Successfully Implemented
Placement	Completed without errors
Routing	Completed without errors
Timing Closure	Achieved
Power Budget	Satisfied

5.9 Final Result Statement

The experimental results demonstrate that the proposed CNN accelerator was successfully implemented and verified on FPGA using Vitis HLS and Vivado. Floorplanning, routing, timing, and power reports confirm that the design meets performance and reliability constraints, making it suitable for real-time embedded acceleration applications.

Chapter 6

DISCUSSIONS

The implementation of the CNN hardware accelerator using Vitis HLS and Vivado demonstrated the practical feasibility of mapping deep learning primitives onto FPGA hardware. The project successfully transformed a high-level C/C++ convolution model into an RTL-based hardware accelerator, showing that computationally intensive operations such as multiply–accumulate (MAC) can be efficiently parallelized in hardware.

The synthesis and implementation results indicate that the architecture achieves a good balance between performance and hardware resource utilization. The use of DSP slices for arithmetic operations significantly improved computational efficiency, while the logic and memory utilization remained within acceptable limits. This shows that FPGA-based acceleration is a suitable approach for real-time and embedded AI applications.

One of the key observations from this project is the effectiveness of hardware pipelining and loop optimization. Applying HLS directives such as pipelining and loop unrolling reduced the overall latency and improved throughput, although these improvements were limited by available hardware resources. The presence of minor resource limitations during synthesis highlights the trade-off between performance optimization and area constraints.

The AXI-based system integration provided a reliable and scalable communication mechanism between the MicroBlaze processor, memory blocks, and the CNN accelerator IP. The successful routing and timing closure confirm that the system is stable at the selected clock frequency, validating the design methodology.

Although the current implementation used a small convolution kernel and limited feature map size, the results clearly demonstrate the scalability potential of the proposed architecture. With further optimization and external memory interfaces, the design can be extended to support deeper CNN layers and larger datasets.

Overall, this work confirms that FPGA-based CNN accelerators designed using high-level synthesis tools offer an efficient and programmable solution for accelerating neural network workloads while maintaining flexibility and low power consumption.

Chapter 7

CONCLUSION

This project successfully demonstrated the design and implementation of a CNN hardware accelerator using high-level synthesis and FPGA-based system integration. A C/C++ based convolution model was converted into an optimized hardware IP using Vitis HLS, and the generated accelerator was integrated into a complete SoC-style system using Vivado. The design incorporated a MicroBlaze soft processor, AXI interconnect, BRAM-based memory architecture, and UART interface.

The hardware design was successfully synthesized, placed, and routed, and post-implementation analysis confirmed that the design met timing and power requirements. The project validated the effectiveness of hardware acceleration techniques such as loop pipelining, loop unrolling, and memory partitioning in significantly improving throughput compared to software-only execution.

The results highlight the suitability of FPGA platforms for implementing compute-intensive neural network operations, offering a good balance between performance, flexibility, and energy efficiency. This work provides a strong foundation for developing scalable and real-time CNN accelerators for embedded and edge AI applications.

Chapter 8

REFERENCES

1. Xilinx Inc., Vitis High-Level Synthesis User Guide (UG1399), AMD Xilinx, 2025.
2. Xilinx Inc., Vivado Design Suite User Guide: High-Level Synthesis (UG998), AMD Xilinx, 2025.
3. Xilinx Inc., Vivado Design Suite User Guide: IP Integrator (UG994), AMD Xilinx, 2022.
4. Xilinx Inc., AXI Reference Guide (UG761), AMD Xilinx, 2025.
5. AMD Xilinx, MicroBlaze Processor Reference Guide (UG984), 2025.
6. Integration of Hardware Acceleration Techniques in a Real Time Framework Using FPGA Device.